Evaluation Chapter: Validation of n8n Workflows

1 Objective

The goal of the evaluation is to systematically compare different validation approaches for n8n workflows. The study investigates which methods reliably detect faulty workflows, from which level of complexity errors begin to be overlooked, and whether non-LLM-based validation procedures can also be used for LLM tasks within a workflow.

2 Core Idea

Each test case consists of:

- a natural-language business description,
- a correct reference workflow,
- several intentionally faulty variants,
- one or more validators.

For each validator, the evaluation measures whether correct and incorrect workflows are classified correctly and which error types are detected or overlooked.

3 Dataset Design

A realistic dataset can consist of `100` base workflows.

Recommended distribution:

- `40` classical sequential business workflows,
- `30` workflows with branching and parallelism,
- `20` more complex exception and approval processes,
- `10` workflows with `1-2` LLM nodes.

For each base workflow, several variants are generated:

- `1` correct workflow,
- `1` structural error variant,

- `1` control-flow error variant,
- `1` semantic error variant,
- for LLM workflows, additionally `1` LLM error variant.

This results in approximately `410` to `500` test objects.

4 Error Classes

4.1 Structural Errors

- missing node,
- wrong connection,
- dead path,
- unreachable node,
- missing merge.

4.2 Control-Flow Errors

- swapped IF branch,
- missing parallelism,
- wrong order,
- incorrectly placed wait or retry step.

4.3 Semantic Errors

- business-wise wrong step,
- missing required step despite formally valid workflow,
- wrong message recipient,
- invoice booked before approval.

4.4 LLM-Related Errors

- LLM node at the wrong position,
- LLM node with the wrong purpose,
- LLM output is accepted without further validation,
- LLM makes a decision that should be checked deterministically.

5 Validators

5.1 Static Validator

Checks:

- JSON structure,
- node types,
- connections,
- reachability,
- dead paths.

Strengths:

- strong for technical and structural errors.

Weaknesses:

- weak for business semantics.

5.2 Rule-Based Business Validator

Checks:

- required steps,
- order,
- branches,
- parallelism,
- target-vs-actual comparison with a reference specification.

Strengths:

- strong for modellable business logic.

Weaknesses:

- limited for open meanings and language variation.

5.3 LLM-as-a-Judge

Input:

- business description,
- workflow JSON or an abstracted workflow view.

Output:

- `valid` or `invalid`,
- explanation,
- optionally error category and confidence.

Strengths:

- better for semantic contradictions.

Weaknesses:

- potentially unstable for subtle structural details,
- more expensive.

5.4 Hybrid

Combines:

- static and rule-based pre-validation,
- LLM-based residual validation for semantic deviations.

Strengths:

- likely the most robust overall.

Weaknesses:

- higher complexity,
- higher cost.

6 Additional Research Question: Complexity

In addition to overall performance, the evaluation investigates whether validation quality decreases with increasing workflow complexity.

Workflow complexity is operationalized through:

- number of nodes,
- number of edges,
- number of branches,
- number of merges,
- control-flow depth,
- number of external integrations,
- number of LLM nodes.

Analysis:

- performance per complexity level,
- errors that are only overlooked from complexity level `x` onward,
- differences between classical workflows and workflows containing LLM nodes.

7 Additional Research Question: LLM Tasks in the Workflow

A separate part of the evaluation examines LLM tasks within a workflow.

The aim is not to validate the full truth of the LLM output. Instead, the following aspects are checked:

- whether the LLM node is placed at the correct business position,
- whether its purpose matches the description,
- whether its output is validated further,
- whether critical decisions are not based blindly on the LLM,
- whether the output flows into the correct branch.

Two approaches are compared:

- `LLM-as-a-Judge` for the entire LLM part,
- a `rule-based approximation` without a judge model.

The rule-based approximation can build on techniques such as those in `validate_n8n_example.ps1`:

- keyword matching,
- stopword filtering,
- start, end, and branch detection,
- checking expected node names,
- comparison of parallelism and alternatives.

This makes it possible to investigate whether, for certain LLM tasks, a relatively simple and explainable validation is already sufficient and where a real judge model remains necessary.

8 Handling External Integrations

An important part of the test setup concerns external systems such as:

- email servers,
- HTTP APIs,
- databases,
- ERP or CRM systems.

For workflow correctness validation, it is generally not sensible to run these integrations by default against real production or live systems.

8.1 Principle

For the main evaluation, external integrations should preferably be `mocked` or `simulated`.

8.2 Reasoning

- Tests become reproducible and controllable.
- Side effects such as real emails or real bookings are avoided.
- Authentication and network problems do not distort the validation.
- Errors can be attributed more clearly to the workflow instead of a third-party system.
- Privacy and security risks are reduced.

8.3 Specifically for Email Servers

For email nodes, mocking or simulation is the most sensible choice for most test cases.

Instead of real delivery, the evaluation can check:

- whether an email would have been sent,
- to which recipient,
- with which subject,
- with which expected business content,
- under which workflow condition.

Possible implementations:

- fake SMTP server,
- local mail catcher,
- test endpoint that logs sending events,
- replacement of the email node by a logging or storage node in a test variant.

8.4 When Real Sending Can Still Be Useful

A small integration sample with real or near-real test infrastructure can additionally be useful to demonstrate external connectivity.

However, this should only play a minor role and should not dominate the main evaluation.

8.5 Methodological Decision

The recommended setup is:

- main evaluation with mocked or simulated integrations,
- optional small integration validation with real test systems.

This ensures that not only structure but also the intended behavior of integration nodes can be validated without compromising scientific comparability through external instability.

9 Test Procedure

For each test object:

1. load the reference description and reference specification,
2. apply the validator to the workflow,
3. store the output:

- `valid` or `invalid`,
- detected errors,
- confidence, if available,
- runtime,
- token cost, if an LLM is used,

4. compare the result with the ground truth.

10 Metrics

Mandatory metrics:

- Accuracy,
- Precision,
- Recall,
- F1,
- False Positive Rate,
- False Negative Rate.

Additional analyses:

- per error class,
- per complexity level,
- for workflows with and without LLM nodes,
- average runtime,
- average token cost.

11 Possible Result Presentations

To ensure that the evaluation does not remain purely textual, the results should be prepared using several complementary representations.

11.1 Tables

- table with overall values per validator: Accuracy, Precision, Recall, F1,
- table per error class: structural, control-flow, semantic, LLM-related,
- table per complexity level: low, medium, high,
- table for workflows with and without LLM nodes,
- table for average runtime and average token cost.

11.2 Bar Charts

- bar chart: `F1` per validator,
- bar chart: `Recall` per error class and validator,
- bar chart: `False Negative Rate` for complex workflows,
- bar chart: comparison of LLM workflows and classical workflows.

11.3 Line Charts

- line: validator performance over increasing node count,
- line: performance over increasing branch count,
- line: recall as a function of workflow depth,
- line: error rate from complexity level `x`.

11.4 Heatmaps

- heatmap: validators by error classes,
- heatmap: validators by complexity levels,
- heatmap: LLM node types by detection rate.

11.5 Boxplots

- boxplot: runtime per validator,
- boxplot: token costs per LLM-based validator,
- boxplot: variation in performance across different domains.

11.6 Confusion Matrices

- confusion matrix per validator for `valid` vs. `invalid`,
- separate matrices for the full set and the LLM subset.

11.7 Case-Based Visualizations

- small workflow figures for selected good and bad examples,
- plus a table with:
  - ground truth,
  - validator decision,
  - detected errors,
  - overlooked errors.

12 Possible Result Lists in the Results Chapter

In the results chapter, findings could be presented roughly in the following order:

1. Overall comparison of all validators
    - Which validator achieves the best overall performance?
    - Which differences are statistically or practically relevant?
2. Results by error class
    - Who detects structural errors best?
    - Who detects control-flow errors best?
    - Who detects semantic errors best?
    - How do validators perform on LLM-related errors?
3. Results by complexity
    - From which node count does performance decrease?
    - From which branching depth are errors frequently overlooked?
    - Which method is most robust for complex workflows?
4. Results for LLM workflows
    - How strongly does validation quality decrease for LLM nodes?
    - Where is the rule-based approximation sufficient?
    - Where is a judge model clearly required?
5. Cost-benefit analysis
    - How much does the hybrid approach improve performance?
    - Does the performance gain justify the additional token cost?
    - Which method is most efficient for practical use?
6. Qualitative error analysis
    - typical missed errors per validator,
    - typical false alarms per validator,
    - concrete examples of edge cases.

13 Expected Non-Trivial Results

It is not expected that a single validator will dominate in all dimensions.
Likely patterns:

- the static validator performs best for structural errors,
- the rule-based validator performs well for clear invariants,

- `LLM-as-a-Judge` performs better for semantic deviations,
- the hybrid approach is overall the most robust but more expensive,
- for LLM tasks, the reliability of all methods decreases, but not equally strongly.

These differentiated differences are what make the evaluation scientifically interesting.

14 Conclusion of the Test Setup

This setup enables:

- a fair comparison of multiple validation approaches,
- an investigation of error detection as a function of complexity,
- a targeted analysis of whether LLM tasks in workflows are partially validatable without a judge model.

This is likely to produce heterogeneous results instead of a trivial statement such as `everything is validatable` or `nothing is validatable`.